



W8 SQL

INSTRUCTOR

Pei-Chi Lo

MIS205

Database Management



Learning Objectives

1. Explain the role of **SQL** in an RDBMS and map a relational schema to SQL
2. Define tables with **DDL**: CREATE / ALTER / DROP TABLE, data types, constraints, and referential integrity
3. Write **SELECT-FROM-WHERE queries** using ORDER BY, BETWEEN, LIKE, DISTINCT, and reason about **NULL** semantics and three-valued logic
4. Combine tables with **JOINS** (implicit, explicit, self-join), and modify data using **INSERT, DELETE, and UPDATE**



Roadmap

- 01 SQL and Schema Definition
- 02 Basic SQL Operators
- 03 Predict The Output
- 04 Weekly Discussion



01

SQL and Schema Definition

Preparation

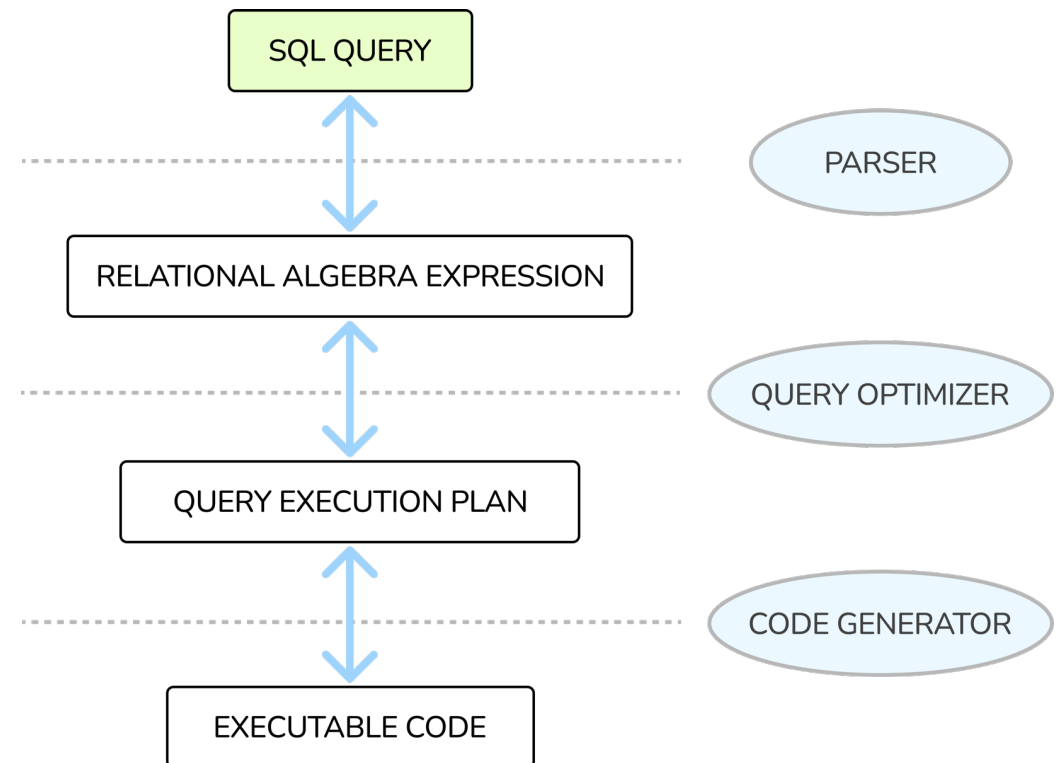
- Download the **example SQL Scripts** from TronClass
- Choose one of the following
 - Download and install Dbeaver
 - Open [sqliteonline](#)
- If you see a beaver
 - It's lab time!



Recap: Relational Query Languages

Languages for **describing queries** in a relational database

- Relational Algebra
 - Intermediate language used within DBMS
 - Procedural/Functional
- **Structured Query Language (SQL)**
 - Predominant application-level query language
 - Declarative



What Is SQL?

SQL stands for **Structured Query Language**

- SQL is a standard language for querying and manipulating data
 - Probably the most successful parallel programming language
 - SQL is a very high-level, **declarative** language
 - SQL commands are **case insensitive**:
 - SELECT = select = Select
 - But `<conditions>` are **case sensitive**



What Is SQL?

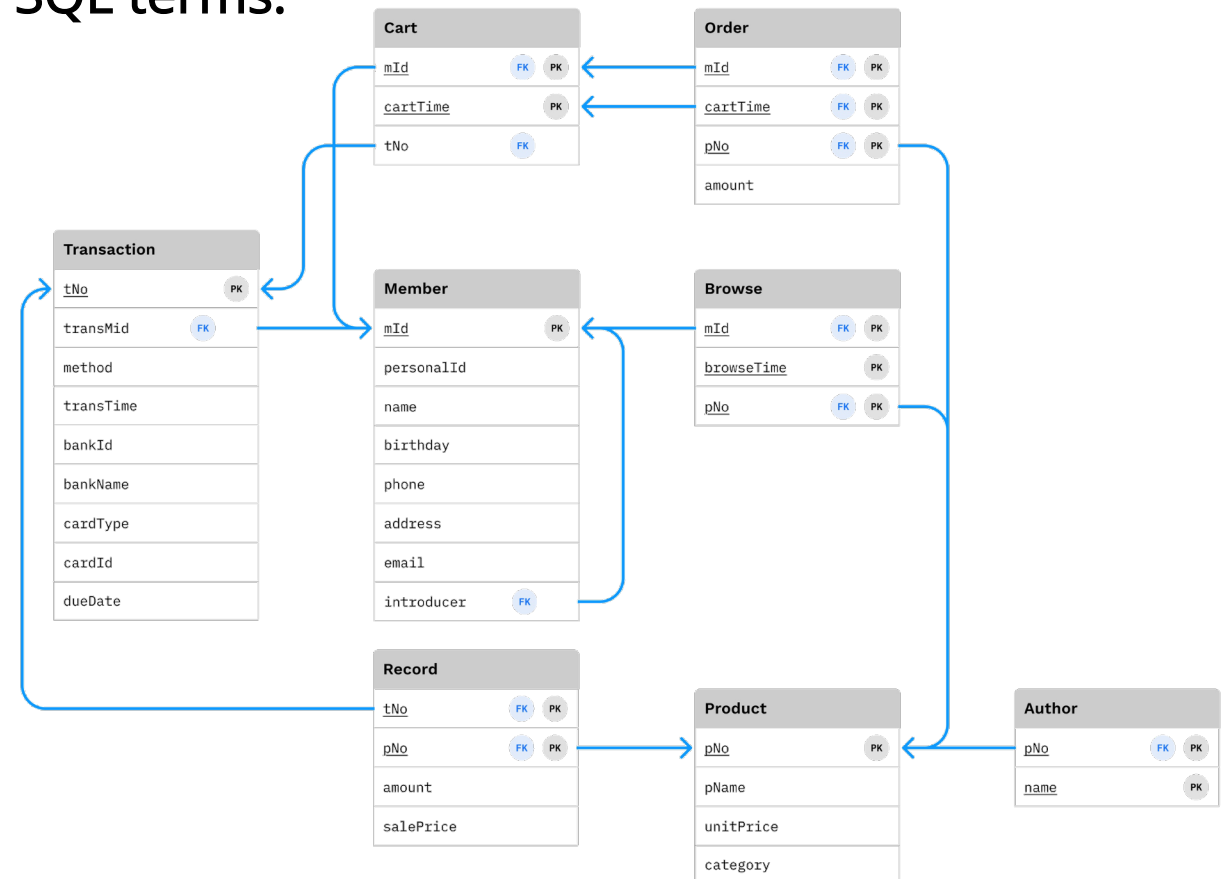
SQL stands for **Structured Query Language**

- Three sub-languages
 - **Data Definition Language (DDL):**
Define schemas; CREATE/ALTER/DROP tables
 - **Data Manipulation Language (DML):**
INSERT/DELETE/UPDATE/SELECT data
 - **Data Control Language (DCL):**
GRANT/REVOKE rights and permissions

From Relational Schema to SQL

Relational model terms map directly to SQL terms:

- Schema → Schema
- Relation → Table
- Attribute → Column
- Tuple → Row
- Domain → Data type



From Relational Schema to SQL

SQL relaxes some rules of the pure relational model:

- SQL allows a **table** to have two or more identical rows
 - No longer a set!
- An SQL **table** is an **ordered** list of tuples
- SQL **relations** can be constrained to be sets by specifying **PRIMARY KEY** or **UNIQUE attributes**, or by using the **DISTINCT** option in a query
- An SQL schema can have **no primary key**. Primary key can be **NULL**.

Schema Concepts in SQL

An SQL **schema** groups related database elements:

- An SQL **schema** is identified by
 1. A **schema** name
 2. An authorization identifier to indicate the user or account who owns the **schema**
 3. Descriptors for each element in the **schema**
 - Schema elements include the **tables**, constraints, views, **domains**, and other constructs that describe the **schema**

Create a schema called STORE, owned by the user with authorization identifier PCLO

```
CREATE SCHEMA STORE AUTHORIZATION PCLO;
```



Data Types in SQL

SQL provides several built-in data types:

- Numeric
 - INT or INTEGER / SMALLINT
 - FLOAT / REAL / DOUBLE PRECISION
 - DEC(i, j) or DECIMAL(i, j) or NUMERIC(i, j)
- Character-string
 - CHAR(n) or CHARACTER(n), where n is the number of characters
 - VARCHAR(n) or CHAR VARYING(n) or CHARACTER VARYING(n), where n is the maximum number of characters
- Bit-string
 - BIT(n) / BIT VARYING(n)



Data Types in SQL

SQL provides types for dates and times:

- Date and Time
 - Date, YYYY-MM-DD, e.g., 2023-04-12
 - Time, hh:mm:ss, e.g., 10:40:31
 - TIME(*i*), hh:mm:ss.ii...i, e.g., 09:12:47.648302
 - *i* additional digits specifying fractions of a second after a period (.) separator
 - TIMESTAMP, YYYY-MM-DD hh:mm:ss, e.g., 2023-04-13 9:30:00
 - INTERVAL, e.g., 5 DAY
 - Specifies a relative value rather than an absolute value
 - Can be DAY/TIME or YEAR/MONTH intervals
 - Can be positive or negative



Data Types in SQL

Large object types for unstructured data:

- Large Data
 - **BLOB** (Binary Large Object)
 - Octal stored bytedata
 - Allows non-printable characters
 - **CLOB** (Character Large Object)
 - Variable length character string



Data Types in SQL

We can specify the domain of each attribute and declare the domain name, e.g.,

```
CREATE DOMAIN PID_TYPE CHAR(10);
```

```
CREATE DOMAIN SALES_TYPE INT CHECK(SALES_TYPE > 100);
```

We can then use `PID_TYPE` in place of `CHAR(10)` later

Basic Operations: **CRUD**

Subsequently, we will learn the four basic SQL operations, known as CRUD:

- Create (**INSERT**)
Add new rows to a [table](#)
- Read (**SELECT**)
Query data from one or more [tables](#)
- Update (**UPDATE**)
Modify existing rows
- Delete (**DELETE**)
Remove rows from a [table](#)

These four operations cover almost everything applications do with a database.

CREATE TABLE

Create the table **Member**

```
CREATE TABLE Member
(mId          CHAR(8)      NOT NULL ,
name         VARCHAR(8)   NOT NULL ,
birthday     DATE,
introducer   CHAR(8),
PRIMARY KEY (mId),
FOREIGN KEY (introducer) REFERENCES Member(mId)
ON DELETE SET NULL ON UPDATE CASCADE);
```



CREATE TABLE

Create the table **Member**

```
CREATE TABLE Member table name
(mId          CHAR(8)      NOT NULL ,
 name         VARCHAR(8)   NOT NULL ,
 birthday     DATE,
 introducer   CHAR(8),
PRIMARY KEY (mId),
FOREIGN KEY (introducer) REFERENCES Member(mId)
ON DELETE SET NULL ON UPDATE CASCADE);
```

CREATE TABLE

Create the table **Member**

CREATE TABLE **Member** table name

attribute
names

(**mId** CHAR(8) NOT NULL ,

name VARCHAR(8) NOT NULL ,

birthday DATE ,

introducer CHAR(8) ,

PRIMARY KEY (**mId**) ,

FOREIGN KEY (**introducer**) REFERENCES **Member**(**mId**)

ON DELETE SET NULL ON UPDATE CASCADE);

CREATE TABLE

Create the table **Member**

CREATE TABLE **Member** table name

attribute
names

(mId	CHAR(8)	NOT NULL,
name	VARCHAR(8)	NOT NULL,
birthday	DATE,	
introducer	CHAR(8),	

data types

PRIMARY KEY (mId),

FOREIGN KEY (introducer) REFERENCES Member(mId)

ON DELETE SET NULL ON UPDATE CASCADE);

CREATE TABLE

Create the table **Member**

CREATE TABLE **Member** table name

attribute
names

(mId	CHAR(8)	NOT NULL,
name	VARCHAR(8)	NOT NULL,
birthday	DATE,	
introducer	CHAR(8),	

data types +
attribute constraints

```
PRIMARY KEY (mId),  
FOREIGN KEY (introducer) REFERENCES Member(mId)  
ON DELETE SET NULL ON UPDATE CASCADE);
```

constraints

Constraints

SQL lets you declare **constraints** that the DBMS enforces on every row:

- Attribute constraints
 - NOT **NULL**: **attribute** must have a value
 - DEFAULT: specify a default value for an **attribute**
 - UNIQUE: no two tuples can have the same value
- Key constraints
 - PRIMARY KEY (**attr**): uniquely identifies each tuple
 - FOREIGN KEY (**attr**) REFERENCES **Table**(**attr**) : referential integrity

Referential Integrity

When a referenced row changes, specify what happens to rows that point to it:

- Let's say we attempt to delete/update a row t with primary key PK :
 - **ON DELETE**
 - **RESTRICT**: **Default Setting**. Only delete when no other rows referring to t
 - **SET NULL**: All foreign keys referring to t updates to **NULL**
 - **SET DEFAULT**: All foreign keys referring to t updates to the default value
 - **CASCADE**: Delete all rows referring to t
 - **ON UPDATE**
 - **RESTRICT**: **Default Setting**. Only update when no other rows referring to t
 - **CASCADE**: Update all foreign keys referring to t

CREATE TABLE

Now let's try to explain this SQL query

CREATE TABLE **Member** table name

attribute names	(mId	CHAR(8)	NOT NULL,	data types + attribute constraints
	name	VARCHAR(8)	NOT NULL,	
	birthday	DATE,		
	introducer	CHAR(8),		

```
PRIMARY KEY (mId),  
FOREIGN KEY (introducer) REFERENCES Member(mId)  
ON DELETE SET NULL ON UPDATE CASCADE);
```

constraints

ALTER TABLE

Add/drop a column from table **Member**

```
ALTER TABLE Member ADD degree VARCHAR(10);
```

```
ALTER TABLE Member DROP address CASCADE;
```

Change the default of a column from table **Order**

```
ALTER TABLE Order ALTER amount SET DEFAULT 100;
```

Add/drop constraints for table **Browse**

```
ALTER TABLE Browse ADD CONSTRAINT... FOREIGN KEY...;
```

DROP TABLE

Remove a table and its definition

Drop table **Member** only if it is not referred by other tables

```
DROP TABLE Product RESTRICT;
```

Drop table **Member** with all references

```
DROP TABLE Product CASCADE;
```



Lab: Set Up Bookstore Database

Steps

1. Open Dbeaver/SQLiteonline and create a new SQLite database connection
2. Open **bookstore_create.sql**, select all (Ctrl+A), then Execute SQL Script
3. Open **bookstore_insert.sql**, select all (Ctrl+A), then Execute SQL Script
4. **Verify:** right-click each table and select View Data to check the records are loaded
5. Explain SQL queries in **bookstore_create.sql**
6. Create Diagram of the database, and observe if you have successfully linked all foreign keys
7. Try any of the **ALTER/DROP** table SQLs in the previous slides



02

Basic SQL Operators

SFW: The Basic SQL Query

Every basic SQL query has the same three-clause form:

SELECT <attributes>

FROM <tables>

WHERE <conditions>;

- Semantics
 - Take cross product $\times$ of all tables in FROM
 - Apply selection σ conditions in WHERE (filtering rows)
 - Apply projection π in SELECT (choosing columns)



SFW: The Basic SQL Query

Selection (**WHERE**): filter rows by condition

```
SELECT * FROM Product WHERE category='Book' ;
```

Projection (**SELECT**): choose which **columns** to return

```
SELECT pName, unitPrice FROM Product WHERE category='Book' ;
```



SFW: The Basic SQL Query

Combine Selection and Projection:

```
SELECT pName, unitPrice  
FROM Product  
WHERE category='Book' AND unitPrice>500;
```



SFW: The Basic SQL Query

LIKE: pattern matching on strings

```
SELECT * FROM Product WHERE pName LIKE '%Theory%';
```

% = any sequence of characters; _ = any single character

DISTINCT: eliminate duplicate rows in results

```
SELECT DISTINCT category FROM Product;
```

Why do we have duplicates in SQL?



SFW: The Basic SQL Query

ORDER BY: sort the result

```
SELECT * FROM Product ORDER BY unitPrice DESC;
```

What are DESC / ASC?

Arithmetic Operations

```
SELECT pName, 0.9*unitPrice FROM Product;
```

BETWEEN: select values in a range

```
SELECT * FROM Product WHERE unitPrice BETWEEN 300 AND 600;
```



NULLs in SQL

Recap: What is *NULL*?

- *NULL* can have multiple meanings
 - Value does not exist
 - Value exists but is unknown
 - Value not applicable

NULLs in SQL

NULL in operations

- Arithmetic with *NULL* : result is always *NULL*
 - If $x = \text{NULL}$, then $4 * (3 - x)$ is still *NULL*
- Boolean logic with *NULL* : three-valued logic
 - $\text{FALSE} = 0 / \text{UNKNOWN} = 0.5 / \text{TRUE} = 1$
 - $C_1 \text{ AND } C_2 = \min(C_1, C_2)$
 - $C_1 \text{ OR } C_2 = \max(C_1, C_2)$
 - $\text{NOT } C_1 = 1 - C_1$



NULLs in SQL

NULL in WHERE clauses

This SQL will not return any attribute=*NULL*:

```
SELECT *
```

```
FROM Product
```

```
WHERE (category='Book') AND (unitPrice<700 AND unitPrice>500);
```

- SQL only returns rows where condition = *TRUE*



NULLs in SQL

Testing for NULL

Finding rows with column having value *NULL* :

```
SELECT *
```

```
FROM Product
```

```
WHERE category='Book' OR category IS NULL;
```

- Test for *NULL*:
 - attribute IS NULL
 - attribute IS NOT NULL



JOINS

A join returns all matching combinations of rows from two or more **tables**

Find **products** under \$600 written by 'Huang'

```
SELECT pName, unitPrice
```

```
FROM Product, Author
```

```
WHERE Product.pNo=Author.pNo
```

 join happens here!

```
AND Author.name='Huang'
```

```
AND unitPrice<600;
```

- Use **TableName.attribute** to clarify ambiguous **attributes**



JOINS

There are different equivalent ways to **join** in SQL

Find products under \$600 written by 'Huang'

```
SELECT pName, unitPrice
FROM Product
JOIN Author
ON Product.pNo=Author.pNo
WHERE Author.name='Huang'
AND unitPrice<600;
```

=

```
SELECT pName, unitPrice
FROM Product, Author
WHERE Product.pNo=Author.pNo
AND Author.name='Huang'
AND unitPrice<600;
```

- We will discuss about this next week...



JOINS

A **table** can be joined with itself using **aliases**

List members introduced by 'b0905555' (level 1)

```
SELECT I.name AS introducer_name,  
       M.mId AS member_id,  
       M.name AS member_name  
FROM Member AS I, Member AS M    Using alias  
WHERE I.mId='b0905555'  
      AND I.mId=M.introducer;
```

- Both **I** and **M** refer to the **Member** table
 - **I** = the introducer, **M** = the introduced member



Lab: Query Challenges

Write SQL queries for each question. Run them in DBeaver

1. List all Book products with price above 500
2. List product names that contain the word 'Theory'
3. List all distinct categories in the Product table
4. List all products sorted by price (highest first)
5. Find the author(s) of 'Database Theory and Practice'
6. List names of members who bought any product
7. List members introduced by Jennifer (b0905555)
8. List members who browsed products priced between 300 and 500

INSERT

Add tuples to a **table**

```
INSERT INTO <table> [<attributes>]
```

```
VALUES <attribute-value-list>;
```



INSERT

Insert a new row to the table **Product**

```
INSERT INTO Product
```

```
VALUES ('b000001', 'DB Mgmt', 700, 'Book');
```

Same order as that of the **attributes** specified in **CREATE TABLE**

Insert a new row to **Product** with **column** name specified

```
INSERT INTO Product (pName, pNo)
```

```
VALUES('DB Mgmt', 'b000001');
```

Attributes not specified get **NULL** values



INSERT

Insert multiple rows from a query

```
CREATE TABLE Book
```

```
(bNo          CHAR(10)      NOT NULL ,  
  bName       VARCHAR(10) ,  
  unitPrice   DECIMAL(10,2));
```

```
INSERT INTO Book
```

```
SELECT pNo, pName, unitPrice  
FROM Product  
WHERE category='Book';
```



Lab: INSERTION

Try running this SQL in DBeaver

```
INSERT INTO Member
```

```
VALUES (NULL, 'PCLO', '2026/01/31', NULL);
```

What happened?



Lab: Look at Example SQL Scripts

Discuss with your group members

1. Explain the SQL queries in `bookstore_insert.sql`

DELETE

Remove tuples from a table

```
DELETE FROM <table>
```

```
WHERE <conditions>;
```

Delete from Product the row with pName valued 'Hero'

```
DELETE FROM Product
```

```
WHERE pName='Hero';
```

Missing WHERE clause deletes **ALL rows!**

UPDATE

Modify attribute values

```
UPDATE <table>
```

```
SET <attribute1>=<value1>, <attribute2>=<value2>, ...
```

```
WHERE <conditions>;
```

Update the unitPrice of a product

```
UPDATE Product
```

```
SET unitPrice=700
```

```
WHERE pNo='b30999';
```

Referential integrity is enforced



Lab: Try These Modifications

Try each statement in DBeaver. Predict what happens first!

1. `UPDATE Product SET unitPrice = 999 WHERE pNo = 'z99999'`

- Verify the change

2. `DELETE FROM Product WHERE pNo = 'b30999'`

- What happens? Why? (Hint: check Record table)

3. `DELETE FROM Product WHERE pNo = 'z99999'`

- Does this work? Why is it different from #2?



03

Predict The Output



Predict the Output: **NULLs**

What does this query return?

```
SELECT *  
FROM Member  
WHERE introducer=NULL;
```

Think about it before running it!

Hint:

What is the result of any comparison with **NULL**?

What should you use instead?



Predict the Output: **NULLs**

What does this query return?

```
SELECT *  
FROM Product  
WHERE unitPrice<500 OR unitPrice>=500 ;
```

Think about it before running it!

Hint:

Why some products are not returned?



Predict the Output: **Duplicates**

Compare these two queries:

1. `SELECT category FROM Product;`

2. `SELECT DISTINCT category FROM Product;`

How many rows does each return? Why are they different?

Hint:

SQL tables are multisets (bags), not sets!

Use `DISTINCT` to get set behavior



Predict the Output: **Empty Table**

Suppose table **S** is empty (0 rows)

```
SELECT DISTINCT R.A  
FROM R, S, T  
WHERE R.A=S.A OR R.A=T.A;
```

What is the result?

Hint:

Remember the semantics:

Step 1: Cross product of R, S, T



04

Weekly Discussion

Submit The SQLs to TronClass

Curate a list of answer SQLs to the queries from lab challenges

- This will be slightly different from what we did before
 - Use LLMs to verify your SQL query
 - **Show the query results of each query**
- Post your revised query/results and notes in the discussion section on TronClass
 - **Week 9 Discussion: SQL**
 - **Due: Next Tuesday, 23:59**



Inquiry →
pclo@mis.nsysu.edu.tw